

Object Relational Mapping

1. What is OrmLite ?

- OrmLite is an Object-Relational Mapper for .NET.
- It maps:
 - C# Class --> MySQL Table
 - Property --> Column
 - C# Object --> Row
- OrmLite also provides APIs for creating tables, inserting, updating, deleting and querying data.

2. What is MySql.Data ?

- MySql.Data is the ADO.NET provider for MySQL. (ADO.NET stands for ActiveX Data Objects for .NET. It is a core **data access technology developed by Microsoft** as part of the **.NET Framework**. It provides a collection of object-oriented classes that act as a communication bridge between .NET applications and various data sources, such as relational databases, XML files, and spreadsheets.)
- It provides low-level database classes such as:
 - MySqlConnection
 - MySqlCommand
 - MySqlReader
- Without OrmLite, you could directly use:
 - C# -> MySql.Data -> MySQL
- With OrmLite:
 - C# -> OrmLite -> MySQL.Data -> MySQL
- OrmLite handles much of the SQL generation and object mapping for us.

3. Create the project and Installation

```
dotnet new console -n OrmLiteDemo
cd OrmLiteDemo

dotnet add package ServiceStack.OrmLite.MySql
```

4. First Connection

- Create a MySQL database:

```
CREATE DATABASE ormlite_demo;
```

- Then:

```
using ServiceStack.OrmLite;
using ServiceStack.OrmLite.MySql;

namespace OrmLiteDemo;

public class Program {

    public static void Main() {

        string connectionString =
"Server=localhost;Port=3306;Database=knowledge_base;User
Id=root;Password=root";

        // comes from ServiceStack.OrmLite.
        // factory: An object whose job is to create/open those connections.
        var dbFactory = new OrmLiteConnectionFactory(
            connectionString,
            // comes from ServiceStack.OrmLite.MySql.
            MySqlDialect.Provider
        );

        // no need for manual dispose if we use `using` keyword.
        // actual db connection object we will use to perform operations.
        // Its type is IDbConnection (interface)
        // OrmLite extends the normal IDbConnection with its database APIs
        through extension methods.
        using var db = dbFactory.Open();

        Console.WriteLine("Connected");
    }
}
```

5. Understand the Code

1. Connection String

```
Server=localhost;Port=3306;Database=knowledge_base;User Id=root;Password=root
```

- It tells the MySQL provider:
 - where MySQL is running
 - which database we need to work with
 - MySQL username
 - MySQL password for authentication

2. OrmLiteConnectionFactory

```
var dbFactory = new OrmLiteConnectionFactory(
    connectionString,
    MySqlDialect.Provider
);
```

- This creates an OrmLite connection factory. It contains connection string + MySQL Dialect.

3. MySqlDialect.Provider

- This tells OrmLite that we are using MySQL.
- OrmLite has different dialect providers for different databases.

4. Open()

- This actually opens the database connection.

5. what is db ?

- This is the object we'll use for almost everything.

```
db.CreateTable<...>();
db.Insert(...);
db.Update(...);
db.Delete(...);
db.Select<...>();
```

6. POCO → Table Mapping

6.1 What is POCO ?

- In simple terms, it is just a normal C# class containing data.

```
public class Student {

    public int Id { get; set; }
```

```

    public string Name { get; set; }
    public int Age { get; set; }
}

```

- There is nothing special here:
 - It does not inherit from OrmLite class.
 - It does not need to implement an OrmLite interface.
 - It is just a normal C# class.
- OrmLite uses this class to understand the database structure.

6.2 Class → Table

- Given:

```

public class Student {

    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
}

```

- OrmLite conventionally interprets it as:

Student

Id	Name	Age

- So:

```

C# class Student
    ↓
MySQL table Student

```

- And:

C# property		MySQL column
-----		-----
Id	→	Id

Name	→	Name
Age	→	Age

- This is convention based mapping.

6.3 Create the table using OrmLite

- Once you have:

```
using var db = dbFactory.Open();
```

- You can write:

```
db.CreateTable<Student>();
```

- OrmLite examines `Student` and generates the corresponding `CREATE TABLE SQL`.
- Conceptually, it generates something similar to:

```
CREATE TABLE STUDENT (  
    Id INT,  
    Name VARCHAR(...),  
    Age INT  
);
```

- The exact generated SQL depends on OrmLite's mapping rules and metadata.
- The important point is:

```
You write C#  
    ↓  
OrmLite examines class  
    ↓  
generates SQL  
    ↓  
MySQL creates table
```

6.4 Does `Id` automatically become the primary key ?

- OrmLite recognizes an `Id` property as the conventional primary key.
- So:

```
public int Id { get; set; }
```

- is treated as the model's primary key.

6.5 Auto-Increment

- If you want MySQL to automatically generate the `Id` :

```
public class Student {  
  
    [AutoIncrement]  
    public int Id { get; set; }  
  
    // rest of attributes  
}
```

- Now during insertion, `Id` is not needed to be supplied.
- This attributes `[AutoIncrement]` comes from `using ServiceStack.DataAnnotations;`

6.6 Complete Example

```
using ServiceStack.DataAnnotations;  
using ServiceStack.OrmLite;  
using ServiceStack.OrmLite.MySql;  
  
namespace OrmLiteDemo;  
  
// another table name then class name we want to use.  
[Alias("students")]  
public class Student {  
  
    [AutoIncrement]  
    [PrimaryKey]  
    public int Id { get; set; }  
  
    // another column name then attribute name we want to use.  
    [Alias("student_name")]  
    // controlling the string length  
    [StringLength(100)]  
    public string Name { get; set; }  
    public int Age { get; set; }  
}  
  
public class Program {
```

```

public static void Main() {

    string connectionString =
"Server=localhost;Database=ormLite_demo;UserID=root;Password=your_password;";

    var dbFactory = new OrmLiteConnectionFactory(
        connectionString,
        MySqlDialect.Provider
    );

    using var db = dbFactory.Open();

    // Student is passed as Generic Type.
    // db.CreateTable<Student>();
    db.CreateTableIfNotExists<Student>();

    Console.WriteLine("Table Created");
}
}

```

6.7 What actually happens internally ?

1. db is our opened OrmLite database connection.
2. OrmLite receives `Student` as the generic type.
3. OrmLite examines the attributes.
4. It examines the metadata as well. (AutoIncrement, Alias, StringLength)
5. The MySQL dialect determines how those definitions should be represented in MySQL.
6. The generated SQL is sent through the underlying MySQL ADO.NET provider.
7. MySQL creates the table.

7. Insert in Table

- `Insert()` says:
 - Insert this object using its mapped fields.
- `InsertOnly()` says:
 - Insert this object, but only use the fields i specify.

8. Save()

- INSERT if not exists
- UPDATE if exists

9. Update()

- `Update()` - Uses primary key to update other entities
- `Update()` with `WHERE` condition - Uses where condition to update other entities
- `UpdateOnly()` - Uses primary key to update entities we tell
- `UpdateOnly()` with `WHERE` condition - Uses where condition to update entities we tell

	WHICH ROWS?	WHAT COLUMNS?
<code>Update</code>	PK / WHERE	Normal model fields
<code>UpdateOnly</code>	PK / WHERE	Explicitly selected fields

10. Delete

- `Delete()` - Uses primary key to delete the entry
- `Delete()` with `WHERE` - Uses where to delete the entry where `WHERE` matches
- `DeleteById()` - Uses primary key (pass only primary key not object)
- `DeleteByIds()` - Uses multiple primary keys (pass array not objects)
- `DeleteAll<T>()` - Executes `DELETE FROM TABLENAME`
- `DropTable<T>()` - Executes `DROP TABLE TABLENAME`

11. Select

- `Select<T>()` - Give me all rows from table [Take in `List<T>`]
- `Select<T>(condition)`
- `Single<T>(condition)` - Returns first instance from result
- `SingleById<T>(Id)` - Used when we know primary key of the record we want
- `Select<T>(condition).First()` - `First()` is LINQ method operating on returned List
- `Exists<T>(condition)` - Does at least one record matching my condition exist ?
- `Count<T>(condition)` - Used when we want to know how many rows match a condition
- `SqlExpression<T> query = From<T>().Where()` then `db.Select(query)`
 - `From<T>()` - Create an OrmLite query expression whose starting table/model is `T`.
 - does not execute immediately
 - it helps build query step by step
 - can execute it later when needed
- `Where()` - Used to add a filter condition to an `SqlExpression<T>`.
 - `Where()` does not executes the query
 - It only builds the query
 - `Where()` itself is a method on the `SqlExpression<T>` query object
- `OrderBy(condition)` and `OrderByDescending(condition)`

- `From<T>().OrderBy(condition)`
- `From<T>().Where(condition).OrderBy()`
- `Take()` - Used to limit the number of rows returned by the query. (LIMIT in SQL)
 - `From<T>().Take(5)`
 - `From<T>().Where().OrderBy().Take()`
- `Skip()` - Tells the DB to skip a certain number of rows before returning the remaining result. (OFFSET in SQL)
 - Not usually used alone, Used with `Take()`
 - `From<T>().Skip(10).Take(10)`
 - `From<T>().OrderBy().Skip(10).Take(5)`
- `From<T>().Select(x => new { x.Name, x.Age })` - Select specific columns only
- `Column<X>`
 - Specifically for the case where our query's result is one column
 - Getting it as `List<T>` is irrelevant if we want only one column. so we can convert it to required `List<X>` using `Column<X>` where `X` is datatype of selected column.
 - `SqlExpression q = db.From<T>().Select(x => x.Name);` - Build the query
 - `List<string> names = db.Column<string>(q);` - Execute the query
- `SelectDistinct()`

```
var query = db.From<T>().SelectDistinct(x => x.City);
List<string> cities = db.Column<string>(query);
// SELECT DISTINCT city FROM TableName;
```

- `ColumnDistinct<T>()`
 - OrmLite also provides dedicated API for this

```
HashSet<string> cities = db.ColumnDistinct<string>(
    db.From<Student>().Select(x => x.City)
);
// SELECT DISTINCT city FROM TableName;
// Column<T>() -> List<T>
// ColumnDistinct<T>() -> HashSet<T>() - collection of unique values
```

12. Aggregate Functions and Scalar<T>

- OrmLite exposes aggregate functions through `Sql.Count`, `Sql.Sum`, `Sql.Avg`, `Sql.Min`, `Sql.Max`, etc.

```
// x is simply the lambda parameter representing one T(Student) object
var countQuery = db.From<Student>().Select(x => Sql.Count("*"));
var sumQuery = db.From<Student>().Select(x => Sql.Sum(x.Age));
```

```
var avgQuery = db.From<Student>().Select(x => Sql.Avg(x.Age));
var minQuery = db.From<Student>().Select(x => Sql.Min(x.Age));
var maxQuery = db.From<Student>().Select(x => Sql.Max(x.Age));
var distinctCityCountQuery = db.From<Student>().Select(x =>
    Sql.CountDistinct(x.City));
```

- Notice that this code **only builds the aggregate queries**.
- The next concept, `Scalar<T>()`, will be used to get output.

Scalar<T>

- Returns one value and convert it into C# type `T`.

```
int a1 = db.Scalar<int>(countQuery);
int totalAge = db.Scalar<int>(sumQuery);
double averageAge = db.Scalar<double>(avgQuery);
```

- `Sql` is class and `Min`, `Max`, `Avg` are its static methods.
- `Sql` class came from `ServiceStack.OrmLite` namespace.

13. Joins

1. Implicit Join

- OrmLite can infer common relationships using its conventions.

```
var query = db.From<Student>().Join<Course>();
```

- The important convention is:
 - `<ReferencedTypeName>Id`
 - Example: `StudentId` in `Course`
 - This joins `Student.Id` and `Course.StudentId`
- OrmLite also has `[References]` attribute. If code does not follow conventions the `[References]` is preferred.

2. Explicit Join

- We can explicitly tell the JOIN condition.

```
var query = db.From<Student>().Join<Student, Course>(
    (student, course) => student.Id == course.StudentId
```

```
);
```

3. Joins

```
// Inner Join
var queryInnerJoin = db.From<Student>().Join<Student, Course>(
    (student, course) => student.Id == course.StudentId
);

// Left Join
var queryLeftJoin = db.From<Student>().LeftJoin<Student, Course>(
    (student, course) => student.Id == course.StudentId
);

// Right Join
var queryRightJoin = db.From<Student>().RightJoin<Student, Course>(
    (s, c) => s.Id == c.StudentId
);

// Full Join
var queryFullJoin = db.From<Student>().FullJoin<Student, Course>(
    (s, c) => s.Id == c.StudentId
);

// Multiple Join
var queryMultipleJoin = db.From<Student>().Join<Student, Course>(
    (s, c) => s.Id == c.StudentId
).Join<Course, Enrollment>(
    (c, e) => c.Id == e.CourseId
);

// Join + Where + Select
SqlExpression<Student> query = db.From<Student>().Join<Student, Course>(
    (student, course) => student.Id == course.StudentId
).Where<Student>(
    student => student.City == "Anand"
).Select<Student, Course>(
    (student, course) => new {
        StudentName = student.Name,
        CourseName = course.Name
    }
);

// StudentCourseDTO is custom class for storing results
```

```
List<StudentCourseDTO> result = db.Select<StudentCourseDTO>(query);
```

14. Group By and Having

```
using System;
using System.Collections.Generic;
using ServiceStack.Data;
using ServiceStack.OrmLite;
using ServiceStack.OrmLite.MySql;

namespace OrmLiteDemo;

public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string City { get; set; }
    public int Age { get; set; }
}

public class CityStudentCountDto
{
    public string City { get; set; }
    public int StudentCount { get; set; }
}

public class Program
{
    public static void Main()
    {
        OrmLiteConnectionFactory dbFactory =
            new OrmLiteConnectionFactory(

"Server=localhost;Database=ormlite_demo;User=root;Password=your_password;",
        MySqlDialect.Provider
        );

        using IDbConnection db = dbFactory.Open();

        db.DropTable<Student>();

        db.InsertAll(new List<Student>
        {
            new Student
```

```
{
    Name = "Jagdish",
    City = "Anand",
    Age = 21
},
new Student
{
    Name = "Rahul",
    City = "Anand",
    Age = 22
},
new Student
{
    Name = "Amit",
    City = "Anand",
    Age = 17
},
new Student
{
    Name = "Raj",
    City = "Ahmedabad",
    Age = 21
},
new Student
{
    Name = "Karan",
    City = "Ahmedabad",
    Age = 23
},
new Student
{
    Name = "Dev",
    City = "Vadodara",
    Age = 20
},
new Student
{
    Name = "Vivek",
    City = "Vadodara",
    Age = 21
},
new Student
{
    Name = "Akash",
    City = "Vadodara",
    Age = 22
}
```

```

    }
});

// -----
// 1. GROUP BY
// Count students in each city
// -----

SqlExpression<Student> groupByQuery =
    db.From<Student>()
        .Select(x => new
        {
            City = x.City,
            StudentCount = Sql.Count("*")
        })
        .GroupBy(x => x.City);

List<CityStudentCountDto> groupByResult =
    db.Select<CityStudentCountDto>(groupByQuery);

Console.WriteLine("1. GROUP BY");

foreach (CityStudentCountDto item in groupByResult)
{
    Console.WriteLine(
        $"{item.City} - {item.StudentCount}"
    );
}

// -----
// 2. GROUP BY + HAVING
// Only cities having at least 3 students
// -----

SqlExpression<Student> havingQuery =
    db.From<Student>()
        .Select(x => new
        {
            City = x.City,
            StudentCount = Sql.Count("*")
        })
        .GroupBy(x => x.City)
        .Having(Sql.Count("*") >= 3);

```

```

List<CityStudentCountDto> havingResult =
    db.Select<CityStudentCountDto>(havingQuery);

Console.WriteLine("\n2. GROUP BY + HAVING");

foreach (CityStudentCountDto item in havingResult)
{
    Console.WriteLine(
        $"{item.City} - {item.StudentCount}"
    );
}

// -----
// 3. GROUP BY + WHERE
// Only students aged 18 or above
// Then group by city
// -----

SqlExpression<Student> whereQuery =
    db.From<Student>()
        .Where(x => x.Age >= 18)
        .Select(x => new
        {
            City = x.City,
            StudentCount = Sql.Count("*")
        })
        .GroupBy(x => x.City);

List<CityStudentCountDto> whereResult =
    db.Select<CityStudentCountDto>(whereQuery);

Console.WriteLine("\n3. GROUP BY + WHERE");

foreach (CityStudentCountDto item in whereResult)
{
    Console.WriteLine(
        $"{item.City} - {item.StudentCount}"
    );
}

// -----
// 4. GROUP BY + WHERE + HAVING
// Students aged 18+
// Group by city

```

```

// Only groups having at least 2 students
// -----

SqlExpression<Student> whereHavingQuery =
    db.From<Student>()
        .Where(x => x.Age >= 18)
        .Select(x => new
        {
            City = x.City,
            StudentCount = Sql.Count("*")
        })
        .GroupBy(x => x.City)
        .Having(Sql.Count("*") >= 2);

List<CityStudentCountDto> whereHavingResult =
    db.Select<CityStudentCountDto>(whereHavingQuery);

Console.WriteLine("\n4. GROUP BY + WHERE + HAVING");

foreach (CityStudentCountDto item in whereHavingResult)
{
    Console.WriteLine(
        $"{item.City} - {item.StudentCount}"
    );
}
}
}

```

15. General Order

- db.From<T>()
- JOIN
- WHERE
- GROUP BY
- HAVING
- SELECT
- ORDER BY
- OFFSET LIMIT

However order does not impact final result in 99% cases i have seen.

```

SqlExpression<Student> query =
    db.From<Student>()
        .Join<Student, Course>(

```

```

        (student, course) =>
            student.Id == course.StudentId
    )
    .Where(student => student.Age >= 18)
    .GroupBy(student => student.City)
    .Having(Sql.Count("*") >= 2)
    .Select(student => new
    {
        City = student.City,
        StudentCount = Sql.Count("*")
    })
    .OrderBy(student => student.City)
    .Skip(5)
    .Take(10);

```

16. Checking the SQL from Expression Query

```

using System;
using ServiceStack.Data;
using ServiceStack.OrmLite;
using ServiceStack.OrmLite.MySql;

namespace OrmLiteDemo;

public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
    public string City { get; set; }
}

public class Program
{
    public static void Main()
    {
        OrmLiteConnectionFactory dbFactory =
            new OrmLiteConnectionFactory(
                "Server=localhost;Database=ormlite_demo;User=root;Password=your_password;",
                MySqlDialect.Provider
            );

        using IDbConnection db = dbFactory.Open();
    }
}

```

```
// -----  
// Query 1  
// -----  
  
var query1 = db.From<Student>()  
    .Where(x => x.Age >= 18)  
    .OrderBy(x => x.Name)  
    .Take(5);  
  
Console.WriteLine("QUERY 1");  
Console.WriteLine(query1.ToSelectStatement());  
  
// -----  
// Query 2  
// Change C# order:  
// Take -> Where -> OrderBy  
// -----  
  
var query2 = db.From<Student>()  
    .Take(5)  
    .Where(x => x.Age >= 18)  
    .OrderBy(x => x.Name);  
  
Console.WriteLine("\nQUERY 2");  
Console.WriteLine(query2.ToSelectStatement());  
  
// -----  
// Query 3  
// OrderBy -> Take -> Where  
// -----  
  
var query3 = db.From<Student>()  
    .OrderBy(x => x.Name)  
    .Take(5)  
    .Where(x => x.Age >= 18);  
  
Console.WriteLine("\nQUERY 3");  
Console.WriteLine(query3.ToSelectStatement());  
  
// -----  
// Query 4  
// Where -> Take -> OrderBy  
// -----
```

```

        var query4 = db.From<Student>()
            .Where(x => x.Age >= 18)
            .Take(5)
            .OrderBy(x => x.Name);

        Console.WriteLine("\nQUERY 4");
        Console.WriteLine(query4.ToSelectStatement());
    }
}

```

17. Raw SQL CRUD

```

using System;
using System.Collections.Generic;
using System.Data;
using ServiceStack.OrmLite;
using ServiceStack.OrmLite.MySql;

namespace OrmLiteDemo;

public class Student
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public int Age { get; set; }
    public string City { get; set; } = "";
}

public class Program
{
    public static void Main()
    {
        // =====
        // 1. CONNECTION
        // =====

        OrmLiteConnectionFactory dbFactory =
            new OrmLiteConnectionFactory(

"Server=localhost;Database=ormlite_demo;User=root;Password=your_password;",
            MySqlDialect.Provider
        );

        using IDbConnection db = dbFactory.Open();
    }
}

```

```

// =====
// 2. CREATE TABLE
// =====
// This is OrmLite's schema API, not raw SQL.
// We're only using it here to prepare the database.

db.DropTable<Student>();

// =====
// 3. RAW SQL INSERT
// =====
//
// ExecuteSql()
//
// Return type:
//     int
//
// Meaning:
//     Number of rows affected.
//
// Use it when:
//     You want to execute INSERT / UPDATE / DELETE
//     or other SQL that doesn't return a result set.
//
// Parameters:
//     new { ... }
//     keeps values separate from SQL.
//
// =====

string insertSql = """
    INSERT INTO Student (Name, Age, City)
    VALUES (@name, @age, @city)
    """;

int rowsInserted = db.ExecuteSql(
    insertSql,
    new
    {
        name = "Jagdish",
        age = 21,
        city = "Anand"
    }

```

```

);

Console.WriteLine($"Rows inserted: {rowsInserted}");

// Insert another student

db.ExecuteSql(
    insertSql,
    new
    {
        name = "Rahul",
        age = 22,
        city = "Ahmedabad"
    }
);

// Insert another student

db.ExecuteSql(
    insertSql,
    new
    {
        name = "Amit",
        age = 19,
        city = "Vadodara"
    }
);

// =====
// 4. RAW SQL SELECT - MULTIPLE ROWS
// =====
//
// SqlList<T>()
//
// Return type:
//     List<T>
//
// Use it when:
//     SQL returns multiple rows and you want each row
//     mapped to a C# object.
//
// =====

```

```

string selectSql = """
    SELECT Id, Name, Age, City
    FROM Student
    WHERE Age >= @minAge
    ORDER BY Name
    """;

List<Student> students = db.SqlList<Student>(<
    selectSql,
    new
    {
        minAge = 18
    }
);

Console.WriteLine("\nStudents:");

foreach (Student student in students)
{
    Console.WriteLine(
        $"{student.Id} - {student.Name} - {student.Age} -
{student.City}"
    );
}

// =====
// 5. RAW SQL SELECT - ONE COLUMN, MULTIPLE ROWS
// =====
//
// SqlColumn<T>()
//
// Return type:
//     List<T>
//
// Use it when your SQL returns ONE column
// but potentially MULTIPLE rows.
//
// =====

string citiesSql = """
    SELECT City
    FROM Student
    """;

List<string> cities =

```

```

        db.SqlColumn<string>(citiesSql);

Console.WriteLine("\nCities:");

foreach (string city in cities)
{
    Console.WriteLine(city);
}

// =====
// 6. RAW SQL SELECT - ONE VALUE
// =====
//
// SqlScalar<T>()
//
// Return type:
//     T
//
// Use it when SQL produces ONE value.
//
// Examples:
//     COUNT(*)
//     SUM(...)
//     AVG(...)
//     MAX(...)
//     MIN(...)
//
// =====

string countSql = """
    SELECT COUNT(*)
    FROM Student
    """;

int studentCount =
    db.SqlScalar<int>(countSql);

Console.WriteLine(
    $"Total students: {studentCount}"
);

// =====
// 7. RAW SQL SELECT WITH AGGREGATE + PARAMETER
// =====

```

```
string averageAgeSql = """
    SELECT AVG(Age)
    FROM Student
    WHERE City = @city
    """;
```

```
decimal averageAge =
    db.SqlScalar<decimal>(
        averageAgeSql,
        new
        {
            city = "Anand"
        }
    );
```

```
Console.WriteLine(
    $"Average age in Anand: {averageAge}"
);
```

```
// =====
// 8. RAW SQL UPDATE
// =====
//
// ExecuteSql() again.
//
// Return type:
//     int
//
// Meaning:
//     Number of rows affected.
//
// =====
```

```
string updateSql = """
    UPDATE Student
    SET Age = @age
    WHERE Name = @name
    """;
```

```
int rowsUpdated =
    db.ExecuteSql(
        updateSql,
        new
        {
```

```

        age = 22,
        name = "Jagdish"
    }
};

Console.WriteLine(
    $"Rows updated: {rowsUpdated}"
);

// =====
// 9. VERIFY UPDATE
// =====

Student? updatedStudent =
    db.Single<Student>(
        x => x.Name == "Jagdish"
    );

Console.WriteLine(
    $"Updated student: {updatedStudent?.Name}, " +
    $"{updatedStudent?.Age}"
);

// =====
// 10. RAW SQL DELETE
// =====
//
// ExecuteSql()
//
// Return type:
//     int
//
// Meaning:
//     Number of rows deleted.
//
// =====

string deleteSql = """
    DELETE FROM Student
    WHERE City = @city
    """;

int rowsDeleted =
    db.ExecuteSql(

```

```

        deleteSql,
        new
        {
            city = "Vadodara"
        }
    );

Console.WriteLine(
    $"Rows deleted: {rowsDeleted}"
);

// =====
// 11. VERIFY FINAL DATA
// =====

List<Student> finalStudents =
    db.SqlList<Student>(
        "SELECT * FROM Student"
    );

Console.WriteLine("\nFinal students:");

foreach (Student student in finalStudents)
{
    Console.WriteLine(
        $"{student.Id} - {student.Name} - " +
        $"{student.Age} - {student.City}"
    );
}
}

```